חניון חכם — תיעוד הפרויקט

מערכת ניהול חניון מבוססת ESP32 עם חיישני אולטרסאונד והקצאת חניות בעזרת A*

גרסת המסמך	2.1
גרסת התוכנה המתועדת	`smart_parking_v9.ino` + `parking_page.h` (דגם שנמדד, אוקטובר 2026)
תאריך	24/10/2026
מגישים	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim
מנחים	Itai Dabran, Ido Ram, Diana Cohen
קוד המקור	github.com/AseelKurdy/smart-parking-lot

1. תקציר מנהלים

המערכת מנהלת חניון חכם בן שתי קומות ו-14 חניות, הבנוי כ**דגם פיזי בקנה מידה 1:100**. כאשר רכב נכנס לחניון, המערכת מקצה לו אוטומטית את החניה הפנויה **הקרובה ביותר לכניסה**, לפי מרחק נסיעה שמחושב באלגוריתם A* על גבי רשת הדרכים של הדגם — במרחקים שנמדדו על הדגם עצמו. אם הרכב חונה בחניה שהוקצתה לו — נורית החניה דולקת באור קבוע; אם הוא חונה בחניה אחרת — הנורית מהבהבת כאתראה.

מתוך 14 החניות, **חניה אחת (מספר 0) ממומשת בחומרה אמיתית** עם חיישן אולטרסאונד ונורית; 13 הנותרות וירטואליות ונשלטות דרך ה-Serial Monitor. חיישן נוסף בכניסה מזהה כל רכב שנכנס.

בנוסף לממשק הטקסטואלי, הלוח מגיש **עמוד אינטרנט גרפי** שמצייר את הדגם עצמו במבט מלמעלה — שתי הקומות, הרמפה וה-U, החניות והרכבים בפרופורציות האמיתיות — ומעדכן אותו בזמן אמת, כולל תצוגת 3D שבה קומה 1 מורמת מעל קומה 0.

2. הגדרת הבעיה והדרישות

2.1 הבעיה

בחניונים גדולים נהגים מבזבזים זמן ודלק בחיפוש אחר חניה פנויה. חניון חכם שיודע אילו חניות פנויות ומכוון כל רכב לחניה ספציפית יכול לקצר את מרחק הנסיעה בתוך החניון.

2.2 דרישות פונקציונליות

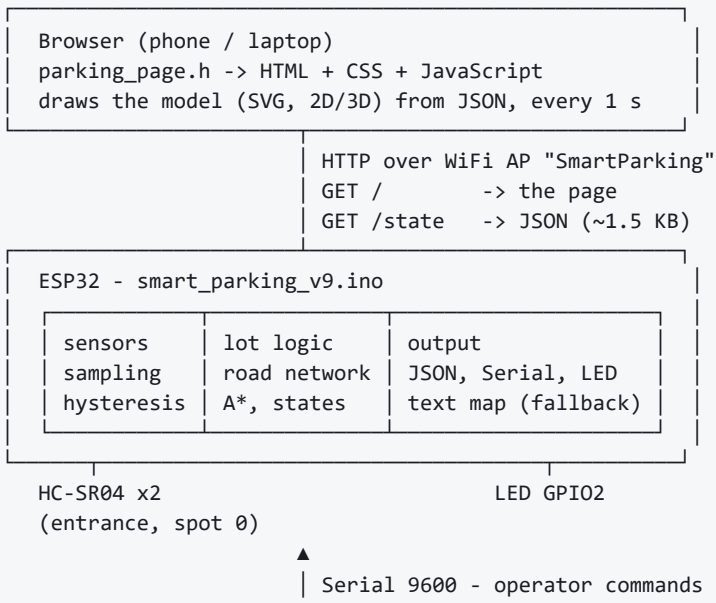
#	דרישה	מימוש
F1	זיהוי רכב הנכנס לחניון (חישה מהצד)	חיישן HC-SR04 בכניסה
F2	זיהוי רכב החונה בחניה (חישה מלמעלה)	חיישן HC-SR04 מעל חניה 0
F3	הקצאת חניה לרכב נכנס	A* על רשת הדרכים — החניה הפנויה הקרובה ביותר
F4	זיהוי חניה במקום שגוי	השוואה בין החניה המשוריינת לחניה בפועל
F5	אתראה על חניה שגויה	נורית מהבהבת (300 ms)
F6	זיהוי מספר רכב	**לא ממומש** בדרישות הפרויקט — מדומה במונה מזהים עולה
F7	חניות וירטואליות נוספות	13 חניות נשלטות דרך ה-Serial Monitor
F8	הדמיה ויזואלית של מצב החניון	עמוד אינטרנט שהלוח מגיש בעצמו, מחקה את הדגם ב-2D/3D

2.3 דרישות לא-פונקציונליות

- המערכת פועלת עצמאית, ללא חיבור לאינטרנט או לשרת חיצוני.
- זמן תגובה: רכב בכניסה ~1.1 שניות (2 דגימות), רכב בחניה 0 ~2 שניות (4 דגימות × 500 ms).
- עמידות בפני רעש חשמלי בקריאות החיישנים.
- הפלט הטקסטואלי באנגלית בלבד — ה-Serial Monitor של Arduino IDE אינו תומך בכיוונויות מימין לשמאל.

3. ארכיטקטורת המערכת

3.1 שכבות



3.2 חלוקת האחריות

רכיב	אחריות
ESP32	דגימת חיישנים, חישוב A*, ניהול מצב החניון, הגשת JSON ו-HTTP
דפדפן	**כל הציור הגרפי** — ה-ESP32 אינו מייצר תמונה, רק נתונים
Serial Monitor	קלט פקודות + יומן אירועים (וציור טקסט כגיבוי כשאין WiFi)
נורית GPIO2	אתראה על חניה 0 בלבד

הפרדה זו נבחרה במכוון: ציור על ה-ESP32 היה צורך זיכרון ו-CPU מיותרים, בעוד שהדפדפן מצייר בחינם מנתונים קומפקטיים.

---

4. החומרה

4.1 רשימת רכיבים

רכיב	כמות	תפקיד
ESP32-WROOM32 DevKit (שבב USB: CH9102)	1	בקר ראשי
HC-SR04 — חיישן אולטרסאונד	2	זיהוי כניסה וזיהוי חניה
Breadboard	2	חיווט
חוטי Jumper	8	חיבור החיישנים
כבל USB	1	הזנה ותקשורת
נורית LED	—	מובנית בלוח (GPIO2)
דגם החניון	1	לוח עם דשא, כביש ראשי, קומה 0, רמפה ומשטח מוגבה (קומה 1)

4.2 חיווט

תפקיד	Vcc	GND	Trig	Echo
חיישן כניסה (מהצד)	VIN (5 V)	GND	**D27**	**D14**
חיישן חניה 0 (מלמעלה)	VIN (5 V)	GND	**D26**	**D25**
נורית אתראה	—	—	—	**GPIO2** (מובנה)

הערה על מתח: ה-HC-SR04 מחזיר פולס Echo ברמת 5 V, ופיני ה-ESP32 מיועדים ל-3.3 V. בפרויקט זה, **לפי אישור מפורש של המנחה**, החיישנים חוברו ישירות ללא מחלק מתח. המערכת פעלה ללא תקלה, אך זו חריגה מהמפרט הרשמי.

4.3 כיול החיישנים (נמדד על הדגם)

חיישן	ללא רכב	עם רכב	הסף בקוד
כניסה (מהצד)	~30 ס"מ (הקיר שמול החיישן)	~22 ס"מ	רכב נספר מתחת ל-**25.0 ס"מ**
חניה 0 (מלמעלה)	25–26 ס"מ	רכב בגובה 2.5 ס"מ: 22.5–23.5; רכב נמוך (1.5 ס"מ): 23.5–24.5	מגיע מתחת ל-**24.7**, יוצא מעל **24.9**

בקוד: `GATE_EMPTY_CM = 30.0`, `GATE_ENTRY_CM = 22.0`, `GATE_CAR_BELOW_CM = 25.0` (סף הזיהוי בכניסה, נקבע ישירות), `SPOT_ARRIVE_CM = 24.7`, `SPOT_LEAVE_CM = 24.9`.

על המרווחים בחניה 0: הספים נבחרו כך שגם רכב נמוך, בגובה 1.5 ס"מ, ייקלט. בין הקריאה הרחוקה ביותר של רכב כזה (24.5) לבין הקריאה הקרובה ביותר בלי רכב (25.0) יש רק 0.5 ס"מ, ולכן נשארים כ-0.2 ס"מ מצד הרכב וכ-0.1 ס"מ מצד החניה הריקה — ברמת הדיוק של ה-HC-SR04. ההיסטרזיס (שני ספים) והדרישה ל-4 דגימות רצופות הם שמונעים קפיצות. אם יש קפיצות, מודדים עם `check 10` ומזיזים את הספים, או מגביהים את החיישן כך שהחניה הריקה תקרא רחוק יותר.

5. מודל החניון

5.1 הדגם



הדגם הפיזי במבט מלמעלה

הכביש הראשי נכנס מלמטה. **קומה 0** היא השטח הפתוח מימין לכביש, עם 7 החניות לאורך הקיר הימני — חניה 0, הקרובה לכניסה, למטה, ועליה חיישן החניה. **הרמפה** מתחילה על הכביש הראשי עוד לפני חניות קומה 0, עולה (קטע דו-סטרי) ומסתובבת ב-U אל **קומה 1** — המשטח המוגבה משמאל, עם 7 החניות לאורך צדו השמאלי; חניה 7 היא הראשונה מכניסת הקומה.

5.2 מידות שנמדדו

גודל	ערך	קבוע בקוד
רכב (אורך × רוחב × גובה)	4 × 2 × 2.5 ס"מ	`CAR_LENGTH_CM` , `CAR_WIDTH_CM` , `CAR_HEIGHT_CM`
חניה (רוחב × אורך)	2.5 × 5 ס"מ	`SPOT_WIDTH_CM` , `SPOT_LENGTH_CM`
כניסה ← ממש לפני החניה הראשונה בקומה 0	22 ס"מ	`ENT_TO_FLOOR0_CM`
כניסה ← תחילת הרמפה	17.5 ס"מ	`ENT_TO_RAMP_CM`
אורך הרמפה (עד הכניסה לקומה 1)	20 ס"מ	`RAMP_CM`
כניסה לקומה 1 ← ממש לפני החניה הראשונה בה	7.5 ס"מ	`FLOOR1_TO_FIRST_CM`

קנה מידה 1:100: חניה של 2.5 × 5 ס"מ בדגם היא בדיוק חניה סטנדרטית של 2.5 × 5 מ', ורכב של 4 × 2 ס"מ הוא רכב של 4 × 2 מ'. לכן כל ס"מ בדגם הוא מטר בחניון אמיתי — אותם מספרים.

5.3 רשת הדרכים

החניון מיוצג כרשת של צמתים וקטעי דרך באורך שנמדד. הרשת נבנית בהפעלה (`buildRoadNetwork`) מהקבועים שלמעלה:

```
         --17.5--> ramp --20--> floor 1 --7.5--> lane 7 --2.5--> lane 8 ... lane 13
        /                                         |5              |5
entrance                                        spot 7          spot 8
        \
         --22--> lane 0 --2.5--> lane 1 --2.5--> ... lane 6
                   |5              |5                  |5
                 spot 0          spot 1              spot 6
```

`lane N` היא הנקודה בנתיב מול חניה N, ו- `floor 1` היא הכניסה לקומה 1 בסוף הרמפה. מכל נקודה בנתיב יש קטע של 5 ס"מ (אורך החניה) לתוך החניה שמולה. אי אפשר לנסוע *דרך* חניה — חניה היא רק יעד.

5.4 מרחקי הנסיעה המתקבלים

חניה	קומה	מרחק מהכניסה	בחניון אמיתי (1:100)
0 (פיזית)	0	27.0 ס"מ	27.0 מ'
1	0	29.5 ס"מ	29.5 מ'
2	0	32.0 ס"מ	32.0 מ'
3	0	34.5 ס"מ	34.5 מ'
4	0	37.0 ס"מ	37.0 מ'
5	0	39.5 ס"מ	39.5 מ'
6	0	42.0 ס"מ	42.0 מ'
7	1	50.0 ס"מ	50.0 מ'
8	1	52.5 ס"מ	52.5 מ'
9	1	55.0 ס"מ	55.0 מ'
10	1	57.5 ס"מ	57.5 מ'
11	1	60.0 ס"מ	60.0 מ'
12	1	62.5 ס"מ	62.5 מ'
13	1	65.0 ס"מ	65.0 מ'

לדוגמה: חניה 0 = 22 (עד מול החניה) + 5 (לתוך החניה) = 27.0; חניה 7 = 17.5 + 20 + 7.5 + 5 = 50.0; וכל חניה נוספת באותה קומה מוסיפה 2.5. ה-A* מחשב את כל הערכים האלה מהרשת, ובדיקה אוטומטית משווה אותם לנוסחה הזו (סעיף 10).

גרסת המחקר (`smart_parking_v9_research`) זהה בדיוק, אבל מציגה את אותם מספרים כמטרים של חניון אמיתי — כמו בסימולציה ובדו"ח.

---

6. אלגוריתם ההקצאה

6.1 הבחירה

נבחר **A*** לפי דרישת הפרויקט. מרחב החיפוש הוא רשת הדרכים: 31 צמתים (כניסה, תחילת רמפה, כניסה לקומה 1, נקודה בנתיב מול כל אחת מ-14 החניות, ובתוך כל חניה) ו-30 קטעי דרך.

6.2 למה רשת ולא טבלה

בגרסאות קודמות החניון היה טבלה (grid) עם צעד קבוע. במדידות של הדגם הרמפה מתחילה *לפני* חניות קומה 0 (17.5 ס"מ מול 22 ס"מ), והקטע הראשון ארוך בהרבה מהמרווח בין החניות — טבלה עם צעד קבוע לא יכולה לייצג את זה. ברשת כל קטע מקבל את אורכו המדוד.

6.3 ההיוריסטיקה

```
h(n) = SPOT_LENGTH_CM                                          if n is not on the goal's lane
h(n) = |position of n on the lane - position of the goal| * SPOT_WIDTH_CM + SPOT_LENGTH_CM
h(goal) = 0
```

הוכחת קבילות (admissibility): כל מסלול לחניה מסתיים בקטע של 5 ס"מ לתוך החניה, ולכן `SPOT_LENGTH_CM` אינו גדול מהמרחק שנשאר. מנקודה על הנתיב של אותה קומה צריך בנוסף לנסוע לאורך הנתיב עד מול החניה — לפחות מספר החניות שביניהן כפול 2.5. לכן h אף פעם אינה מעריכה ביתר, ו-A* מחזיר את המסלול הקצר ביותר.

6.4 מימוש

- **החישוב מתבצע פעם אחת.** הדגם קבוע, ולכן `computeSpotDistances()` בונה את הרשת ומריצה A* ל-14 היעדים ב- `setup()`. בזמן ריצה לא מתבצע חיפוש.
- **אין סגירת צמתים (closed set):** צומת יכול להיפתח מחדש אם נמצא לו מסלול זול יותר. ברשת בת 31 צמתים העלות זניחה.
- **הערה על יעילות:** כיוון שהכניסה קבועה, ריצה אחת של Dijkstra הייתה מחזירה את כל המרחקים בבת אחת. ההבדל זניח, ו-A* נבחר כפי שהוגדר בדרישות.

---

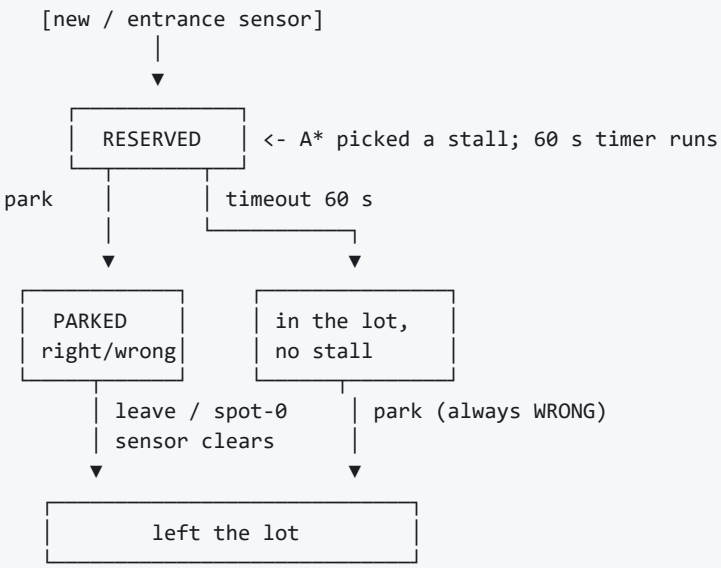
7. מודל המצב

7.1 מצבי חניה

מצב	קוד	משמעות
`free`	`SP_FREE`	פנויה וזמינה להקצאה
`RESERVED`	`SP_RESERVED`	הוקצתה לרכב שעדיין בדרך אליה
`OCCUPIED`	`SP_OCCUPIED`	רכב חונה בה בפועל
`BLOCKED`	`SP_BLOCKED`	**רק לחניה 0** — החיישן מזהה עצם, אך אין רכב רשום

סדר העדיפויות: תפוסה ← משוריינת ← חסומה ← פנויה. מצב `BLOCKED` נדרש משום שהחיישן אינו יודע **מי** נמצא בחניה; חפץ שמונח שם בלי שרכב הוקצה לא משויך לאף רכב, והחניה אינה מוקצית עד שתתפנה.

7.2 מעגל החיים של רכב



7.3 מנגנון השריון

כשרכב נכנס, החניה שהוקצתה לו מסומנת `RESERVED` ואינה מוצעת לרכבים אחרים (תיקון לבאג מוקדם שבו כל רכב חדש קיבל את אותה חניה). השריון פוקע לאחר 60 שניות; הרכב נשאר בחניון ויכול עדיין לחנות, אך כל חניה שיתפוס תסומן כשגויה.

7.4 השתלטות על חניה משוריינת

אם רכב א' חונה בחניה שהוקצתה לרכב ב', המערכת **אינה דוחה את הפעולה** — פיזית אי אפשר למנוע זאת. החניה נרשמת על רכב א' (`WRONG SPOT`), ורכב ב' מקבל הקצאה חדשה (`[REROUTE]`); אם אין חניה פנויה, רכב ב' יוצא מהחניון.

---

8. מנגנוני עמידות ברעש

תופעה	פתרון	קבוע
פולסים חשמליים קצרים על קו ה-Echo (0.0–0.1 ס"מ)	קריאה מתחת ל-2 ס"מ נפסלת	`MIN_VALID_CM = 2.0`
הפרעה הדדית בין שני החיישנים	השהייה של 60 ms בין המדידות	`CROSSTALK_DELAY_MS = 60`
ריצוד סביב סף בחניה 0	שני ספים (היסטרזיס)	`SPOT_LEAVE_CM` / `SPOT_ARRIVE_CM`
קריאה בודדת שגויה בחניה 0	4 דגימות רצופות ועקביות	`SPOT_CONFIRM_SAMPLES = 4`
רכב עקמומי שמקפיץ את קריאת הכניסה	2 דגימות רצופות לאישור, 2 "ריק" לפני רכב הבא	`GATE_CONFIRM_SAMPLES`, `GATE_CLEAR_SAMPLES`
אירוע שווא ברגע ההפעלה	הדגימה הראשונה רק מאתחלת מצב	`firstSample`

דגימה לא תקינה בחניה 0 **מאפסת** את מונה האישורים — הרעש מגיע בפרצים.

---

9. מהלך הפיתוח

גרסה	שינוי עיקרי
v1	קריאת שני חיישנים והדפסת מרחקים
v2	היסטרזיס, מזהי רכב מפורשים בפקודות, פקודת `check`
v3	תיקון שגיאת קומפילציה (`struct` בחתימת פונקציה) — הוחלף במערכים
v4	גישת החיישנים שנבדקה בפועל: דגימה כל 500 ms, השהיית cross-talk
v5	ביטול הדפסה רציפה; המרחקים רק דרך `check`
v6	**מנגנון שריון**, רכב אחד–חניה אחת
v7	`park <spot> <id>` , `leave <id>` לפי מזהה רכב
v8	יציאה בחיישן חניה 0, פקיעת שריון, ניתוב מחדש, המלצה לרכב הבא, טיפול ברעש, מצב `BLOCKED`
v9	עמוד אינטרנט גרפי, `status` מרענן את התמונה, רשימת רכבים ממתינים
v9 (דגם שנמדד)	**מרחקים ומידות שנמדדו על הדגם (ס"מ), רשת דרכים ו-A* חדש, כיול חדש לשני החיישנים, תמונה שמחקה את הדגם ב-2D/3D**

9.1 באגים משמעותיים שנמצאו ותוקנו

1. **Trig ו-Echo הוחלפו פיזית** בחיישן הכניסה — תוקן בקוד, על ידי החלפת מספרי הפינים.
2. **חיישן החניה נתקע לנצח** — רצועת היסטרזיס משותפת מיקמה את קריאת החניה הריקה בתוך האזור המת. תוקן בספים נפרדים.
3. **אותו רכב חנה בשתי חניות** — נוספה בדיקה `whereIsCarParked()` .
4. **כל רכב חדש קיבל את אותה חניה** — נוסף מצב `RESERVED` .
5. **"רכבי רפאים" בהפעלה** — תוקן ב- `firstSample` .
6. **רכבי רפאים מרעש בחניה 0** — חיישן החניה אינו יוצר רכבים, ורעש מאפס את האישור.

---

10. בדיקות

כל הבדיקות נמצאות בתיקייה `tests/` ורצות על המחשב, בלי הלוח (פירוט ב- `tests/README.md`).

10.1 איך בודקים בלי חומרה

הקובץ `smart_parking_v9.ino` **עצמו**, בלי שינוי, מקומפל על המחשב יחד עם תחליפים לחומרה: פורט סריאלי, שעון, שני חיישני ה-HC-SR04 (כולל מודל רעש), הנורית ושרת האינטרנט. כך אפשר להזיז "רכב" מול חיישן, לקרוא את הנורית ולבדוק את ה-JSON שהעמוד מקבל.

10.2 תוצאות

בדיקה	מה נבדק	תוצאה
חבילת הבדיקות (`run_tests.ps1 -Sim`)	38 הבדיקות המקוריות + 16 חדשות: חיישנים, נורית, נתוני התמונה	**54 מתוך 54 עברו**
בדיקת מודל אקראית (`fuzz_model.py`)	אחרי כל פעולה, מצב הקושחה מול מימוש עצמאי של הכללים	**161,040 פעולות, 0 הבדלים**
מקרי קצה (`edge_cases.py`)	מזהים תלת-ספרתיים, יותר מ-16 רכבים ממתינים, גודל ה-JSON	**13 מתוך 13**
קומפילציה ל-ESP32 (`esp32_syntax_check.ps1`)	המהדר והספריות האמיתיים של ESP32	**עובר, בלי אזהרות** (גם גרסת המחקר)
עמוד האינטרנט בדפדפן	מאות מצבים אמיתיים מהקושחה; כל רכב, סימון ומונה בציור מול הנתונים	**1,029 מצבים, 0 הבדלים**

בין הבדיקות: כל 14 המרחקים מול הנוסחה של סעיף 5.4; סף הכניסה בדיוק ב-25 ס"מ (24.9 = רכב, 25.1 = ריק); ספי חניה 0 מול טווחי המדידה (25.0 ו-26.0 אינם רכב; 24.5 — רכב נמוך — ו-23.5 הם רכב; ותחום 24.7–24.9 אינו משנה מצב); רכב שעומד מול חיישן הכניסה נספר פעם אחת; רעש ואי-הד אינם יוצרים אירועים; ונורית קבועה / מהבהבת.

על הלוח עצמו: חבילת 38 הבדיקות רצה על הלוח האמיתי ב-10/10/2026 (גרסת v8) ועברה במלואה, 38 מתוך 38. הגרסה הנוכחית רצה על הלוח בהדגמה המצולמת ב-18/10/2026 (הסרטון): זיהוי רכב בכניסה, חיישן חניה 0, שריון שפג, חניה במקום לא נכון, הנורית ועמוד האינטרנט שמוגש מה-ESP32.

10.3 מה לא נבדק אוטומטית

חבילת הבדיקות האוטומטית עוד לא הורצה מחדש על הלוח עם הגרסה הנוכחית (ההתנהגות על הלוח נבדקה ידנית בהדגמה המצולמת). מריצים אותה כך: `run_tests.ps1 -Port COMx` (ראו `tests/README.md`).

---

11. מגבלות ידועות

מגבלה	השלכה
חיישן חניה פיזי יחיד	רק חניה 0 מזוהה אוטומטית; 13 הנותרות דרך פקודות
מרווח צר בחניה 0	~0.2 ס"מ מצד רכב נמוך ו-~0.1 ס"מ מצד החניה הריקה (סעיף 4.3)
אין זיהוי רכב אמיתי	המזהה הוא מונה עולה; רכב שמגיע לחניה 0 משויך בהנחה
רכב ששריונו פג	נשאר ברשימת הממתינים עד `leave`
`MAX_CARS = 500`	לאחר 500 רכבים נדרשת הפעלה מחדש
ציור הטקסט ב-9600 baud	כ-1.4 שניות לציור, כשה-WiFi כבוי
העמוד הגרפי לצפייה בלבד	כל השליטה דרך ה-Serial Monitor
אין שמירה מתמדת	כיבוי הלוח מאפס את מצב החניון

12. פרויקטים דומים שנסקרו

פרויקט	רלוונטיות
Kadacheahmedrami/smart-parking	6 חיישני HC-SR04 + לוח בקרה. אותם פינים שנבחרו כאן
Muhaiminul-Hasan/RFID-based-Smart-Parking-System-using-ESP32	זיהוי RFID מול טבלה — התבנית למעבר לזיהוי אמיתי
AHD184/esp32-freertos-smart-parking-system	FreeRTOS, RFID, שער servo; החיישנים מזהים אנומליות
AmerWegzyy/IOT_ParkMe	HC-SR04 מפעיל ESP32-CAM; רעיון לאימוץ: כיול סף בלחיצת כפתור

אף אחד מהפרויקטים האלה אינו ממש את הכלל *חניה נכונה* ← *אור קבוע, חניה שגויה* ← *הבהוב* — זו התרומה המקורית של הפרויקט.

---

13. עבודה עתידית

1. **העלאה ללוח והרצת החבילה על החומרה**, עם ציור הטקסט מופעל (`TEXT_MAP = true`).
2. **כיול בלחיצת כפתור** — שמירת המרחק הנמדד כסף, במקום עריכת קבועים.
3. **RFID אמיתי** (MFRC522) במקום מונה המזהים.
4. **חיישנים נוספים** לחניות נוספות — המבנה תומך בכך דרך `REAL_SPOT_INDEX` .
5. **חיווט טוב יותר לחיישן החניה** — הפרדת חוטי Trig/Echo וקבל 10 µF על ההזנה, נגד רעש חשמלי.

---

14. נספחים

נספח א' — קבצי הפרויקט

קובץ / תיקייה	תיאור
`smart_parking_v9/smart_parking_v9.ino` (ב-GitHub: `firmware/smart_parking_v9/`)	הקושחה — חיישנים, רשת הדרכים ו-A*, לוגיקת החניון, שרת HTTP
`smart_parking_v9/parking_page.h`	עמוד האינטרנט (HTML/CSS/JS) — חייב לשבת באותה תיקייה
`smart_parking_v9_research/`	אותה קושחה, מרחקים במטרים בקנה מידה אמיתי
`tests/`	סימולטור, חבילת הבדיקות, בדיקת מודל, מקרי קצה, קומפילציה ל-ESP32
`simulation/`	סימולציית החניון בגודל אמיתי (Python) לדו"ח
`report/`	הדו"ח (מאמר) והסקריפטים שבונים אותו
`docs/`	בניית ה-PDF של שלושת המסמכים ותמונות

נספח ב' — מילון מונחים

מונח	הסבר
שריון (Reservation)	הקצאת חניה לרכב שטרם הגיע אליה
היסטרזיס	שני ספים שונים למעבר בין מצבים, למניעת ריצוד
Cross-talk	הפרעה הדדית בין שני חיישני אולטרסאונד
היוריסטיקה קבילה	הערכה שאינה מעריכה ביתר את המרחק שנשאר
AP (Access Point)	הלוח פותח רשת WiFi משלו
קנה מידה 1:100	ס"מ אחד בדגם = מטר אחד במציאות